Cicada 2067 - Database Schema (PostgreSQL & Supabase)

1. users

Single unified table serving as the pre-authorized whitelist and user profile. Supabase Auth will use a PostgreSQL Trigger to verify the email against this table before allowing login.

id (UUID)	email (VARCHAR)	display_name (VARCHAR)	role (ENUM)	team_id (UUID)	joined_team_at (TIMESTAMPTZ)	created_at (TIMESTAMPTZ)
(empty)	*(empty)*	*(empty)*	`participant/admin`	*(empty)*	*(empty)*	*(empty)*

2. teams

Created by a Team Leader. Generates a unique invite code for members (Max 5 per team).

id (UUID)	name (VARCHAR)	leader_id (UUID)	invite_code (VARCHAR)	current_round (INT)	current_challenge_id (UUID)	challenges_completed (INT)	total_time_taken (INT)	is_disqualified (BOOL)
(empty)	*(empty)*	*(empty)*	*(empty)*	*(empty)*	*(empty)*	*(empty)*	*(empty)*	*(empty)*

3. challenges

Unified challenges table holding the actual puzzles, storylines, and assets across all rounds.

id (UUID)	round_number (INT)	sequence_number (INT)	title (VARCHAR)	story_context (TEXT)	embedded_assets (JSONB)	answer (VARCHAR)	unlocks_story_fragment (TEXT)
(empty)	*(empty)*	*(empty)*	*(empty)*	*(empty)*	*(empty)*	*(empty)*	*(empty)*

4. submission_logs

Critical for admin tracking, analytics, and resolving disputes.

id (UUID)	team_id (UUID)	user_id (UUID)	challenge_id (UUID)	submitted_answer (VARCHAR)	is_correct (BOOL)	submitted_at (TIMESTAMPTZ)
(empty)	*(empty)*	*(empty)*	*(empty)*	*(empty)*	*(empty)*	*(empty)*

5. system_config

Global State for Admin God-Mode (e.g. pause rounds, lock system).

key (VARCHAR)	value (JSONB)	updated_at (TIMESTAMPTZ)
(empty)	*(empty)*	*(empty)*

Core Security & Role Segregation (Supabase RLS)

Supabase utilizes **Row Level Security (RLS)** directly on the PostgreSQL database to ensure absolute security. Roles are defined by the `role` ENUM in the `users` table.

Participant Privileges (role = 'participant')

- **`users`**: Can `SELECT` their own row and rows of users with the same `team_id`.
- **`teams`**: Can `SELECT` their own team. Can `UPDATE` only if they are the `leader_id`.
- **`challenges`**: Can `SELECT` only the challenge where `id = team.current_challenge_id`. Cannot see future challenges.
- **`submission_logs`**: Can `INSERT` a submission. Cannot `SELECT` logs to prevent reverse engineering.
- **Answers**: The `answer` column is NEVER sent to the frontend. Validation occurs entirely via backend secure functions.

Administrator Privileges (role = 'admin')

- **Full Access**: Admins completely bypass RLS policies. They have unrestricted `SELECT`, `INSERT`, `UPDATE`, and `DELETE` access to all tables.
- **God-Mode**: Can manually modify `teams.current_challenge_id` to skip puzzles.
- **Global Configuration**: Can modify `system_config` to pause the timer globally or lock out submissions.
- **Analytics**: Can query the entire `submission_logs` table for real-time leaderboards.